

**ON FORMAL VERIFICATION TECHNIQUES FOR MASKED
CIRCUITS: APPROACHES AND THEORETICAL
UNDERPINNINGS**

by
KAĞAN GÜLSÜM

Submitted to the Graduate School of Engineering and Natural Sciences
in partial fulfilment of
the requirements for the degree of Master of Science
in Computer Science and Engineering

Sabancı University
July 2026

**ON FORMAL VERIFICATION TECHNIQUES FOR MASKED
CIRCUITS: APPROACHES AND THEORETICAL
UNDERPINNINGS**

Approved by:

Asst. Prof. SÜHA ORHUN MUTLUERGİL
(Thesis Supervisor)

Prof. ERKAY SAVAŞ

Asst. Prof. KLAUS FREIHERR VON GLEISSENTHAL

Date of Approval: July 22, 2026

KAĞAN GÜLSÜM 2026 ©

All Rights Reserved

ABSTRACT

ON FORMAL VERIFICATION TECHNIQUES FOR MASKED CIRCUITS: APPROACHES AND THEORETICAL UNDERPINNINGS

KAĞAN GÜLSÜM

Computer Science and Engineering, M.Sc. Thesis, July 2026

Thesis Supervisor: Asst. Prof. SÜHA ORHUN MUTLUERGİL

Keywords: keyword1, keyword2, keyword3, keyword4

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

ÖZET

MASKELENMİŞ DEVRELER İÇİN FORMAL DOĞRULAMA TEKNİKLERİ ÜZERİNE: YAKLAŞIMLAR VE KURAMSAL TEMELLER

KAĞAN GÜLSÜM

Bilgisayar Bilimi ve Mühendisliği, Yüksek Lisans Tezi, Temmuz 2026

Tez Danışmanı: Dr. Öğr. Üyesi SÜHA ORHUN MUTLUERGİL

Anahtar Kelimeler: kelime1, kelime2, kelime3, kelime4

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

ACKNOWLEDGEMENTS

I would like to thank my advisor and family.

Dedicated to Okan Buruk...

TABLE OF CONTENTS

ABSTRACT	iv
ÖZET	v
LIST OF TABLES	xii
LIST OF FIGURES	xiii
LIST OF SYMBOLS	xiv
LIST OF ABBREVIATIONS	xv
1. INTRODUCTION	1
1.1. Side-Channel Attacks and the Masking Countermeasure.....	1
1.2. Verifying Masked Implementations	2
1.3. Scope and Threat Model	2
1.4. Contributions.....	3
1.5. Thesis Outline	4
2. PRELIMINARIES	5
2.1. Notation and Mathematical Background	5
2.1.1. The Field $\mathbb{F}_2$ and Multilinear Boolean Polynomials.....	5
2.1.2. Probability Distributions and Statistical Independence.....	5
2.1.3. Boolean Circuits and Gates	5
2.2. Secret Sharing and Masking.....	7
2.2.1. Boolean Masking	7
2.3. The ISW Probing Model	7
2.3.1. t -Threshold Probing and Observation Sets.....	7
2.3.2. d -Probing Security	8
2.3.3. Probabilistic Non-Interference	8
2.3.4. Threshold and Strong Non-Interference	8

2.4. Randomness Substitution	8
2.5. Computational Hardness Preview	10
3. RELATED WORK	11
3.1. Language-Based Verification	12
3.1.1. Verified Proofs of Higher-Order Masking	12
3.1.2. Large Sets and the Exploration Algorithm.....	12
3.1.3. Witness Replay and Derivation Reuse	12
3.1.4. MaskVerif With Physical Defaults	12
3.2. Fourier-Analytic Verification	12
3.2.1. The Xiao-Massey Criterion and Approximations	12
3.2.2. SAT-Based Realization and Its Cost	12
3.3. Other Approaches	12
3.3.1. Model Counting and SMT	12
3.3.2. Direct Linear-Algebraic Methods	12
3.3.3. Compositional Reasoning and Relabeling	12
3.4. Position of This Work	12
4. COUPLINGS AND PROBING SECURITY	13
4.1. The Counting Problem Is #P-Complete	14
4.1.1. From Independence to Distribution Counting	14
4.1.2. #P-Completeness and Its Consequences for Verification.....	14
4.2. Couplings of Probabilistic Programs	14
4.2.1. Coupling Two Runs Under Different Secrets	14
4.2.2. Coupling Functions: Randoms to Randoms.....	14
4.3. Probing Security via Couplings	14
4.3.1. d -Probing Security Implies a (Family of) Coupling Functions ..	14
4.3.2. Identity-On-Leakage Couplings and Secret-Indistinguishability	14
4.4. Optimistic Sampling as Coupling Construction	14
4.4.1. Rewrites as Bijections on the Random Space	14
4.4.2. The Derivation as an Explicit Coupling Witness	14
4.4.3. Why a Successful Rewrite Yields a Valid Coupling	14
5. DESIGN OF XXX	15
5.1. Pipeline Overview	15

5.2. Expression Representation	15
5.2.1. A Hash-Consed n -Ary Circuit DAG	15
5.2.2. Canonicalization: Flattening and Cancellation	15
5.2.3. Multiplicative Factoring.....	16
5.2.3.1. The Factoring Transformation.....	16
5.2.3.2. Enlarging the Class of Verifiable Circuits	16
5.2.3.3. Cost Analysis and the Factoring-Speed Tension	16
5.3. Component I: The Rewrite Engine	16
5.3.1. The Simple Rewrite Rule	17
5.3.2. Fallback: General Substitution.....	19
5.3.3. Relation to the Coupling-Construction View	21
5.4. Component II: Traversing the Subset Space.....	23
5.4.1. The Space-Splitting Worklist Representation	23
5.4.2. Large-Set Pruning and Closure-Based Splitting.....	24
5.4.3. Completeness via Vandermonde’s Identity	24
5.5. Component III: Extending Probe Sets	24
5.5.1. Closures: Structural Deduction of Free Wires	24
5.5.2. Coupling-Driven Extension	24
6. IMPLEMENTATION OF XXX	25
7. EVALUATION	26
7.1. Benchmarks	26
7.2. Methodology and Metrics	26
7.3. Results	26
7.3.1. Verification Outcomes and Case Studies	26
7.3.2. Redundancy in the Search	26
7.3.3. The Cost and Benefit of Extending Probe Sets	26
7.4. Discussion	26
8. FUTURE WORK	27
8.1. Arithmetic Circuits.....	27
8.2. Symmetry-Orbit Space Exploration	27
8.3. Randomized Fast-Reject for the Engine.....	27
8.4. Scaling and Compositional Verification	27

8.5. Glitches and Practical Aspects	27
9. CONCLUSION	28
BIBLIOGRAPHY.....	29
APPENDIX.....	30

LIST OF TABLES

LIST OF FIGURES

LIST OF SYMBOLS

- Φ Coupling function: a family $\{\Phi_{\mathbf{a},\mathbf{b}}\}$ of measure-preserving bijections of the random space, one per ordered pair of secret assignments.
- d Probing order (maximum number of wires an adversary may observe).
- σ Randomness substitution $r \mapsto e \oplus r$; its induced measure-preserving map on the random space is written $\hat{\sigma}$.

LIST OF ABBREVIATIONS

ISW Ishai, Sahai, Wagner.

1. INTRODUCTION

1.1 Side-Channel Attacks and the Masking Countermeasure

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

1.2 Verifying Masked Implementations

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

1.3 Scope and Threat Model

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante,

dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

1.4 Contributions

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur

ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

1.5 Thesis Outline

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

2. PRELIMINARIES

2.1 Notation and Mathematical Background

filler

2.1.1 The Field $\mathbb{F}_2$ and Multilinear Boolean Polynomials

filler

2.1.2 Probability Distributions and Statistical Independence

filler

2.1.3 Boolean Circuits and Gates

We start, as one should, with the object the rest of the thesis is built on. The definition is the standard digraph formulation of a circuit, specialised to arithmetic over the two-element field.

Definition 2.1 (Arithmetic circuit over $\mathbb{F}_2$). *Fix the field $\mathbb{F}_2 = \{0, 1\}$ and a set $S = \{s_i\}$ of gate functions, each of some arity $a_i \in \mathbb{N}_{\geq 1}$, i.e. each s_i maps $\mathbb{F}_2^{a_i}$ to*

$\mathbb{F}_2$. A circuit C is a finite directed graph (a digraph) containing no directed cycle, such that:

- Every vertex of indegree 0 is labelled either by an indeterminate, in which case it is an input gate, or by an element of $\mathbb{F}_2$, in which case it is a constant gate. The input gates are partitioned into secret inputs $\mathcal{S}$, random inputs $\mathcal{R}$, and public inputs $\mathcal{P}$.
- Every remaining vertex is a computation gate, labelled by a function $s_i \in S$ whose arity a_i coincides with the vertex's indegree. A computation gate of outdegree 0 is an output gate.

We call the vertices wires, and for a computation gate we speak interchangeably of the gate and the wire carrying its output.

Over $\mathbb{F}_2$ there is no difference between the Boolean and the arithmetic reading of such a circuit: exclusive-or is addition, logical-and is multiplication, and the two together already express every function $\mathbb{F}_2^n \rightarrow \mathbb{F}_2$. So we fix

$$S = \{\oplus, \odot\},$$

addition and multiplication in $\mathbb{F}_2$ (Section 2.1.1), both of arity 2. Every computation gate then has indegree exactly 2 and reads

$$w_i = w_j \oplus w_k \quad \text{or} \quad w_i = w_j \odot w_k.$$

Throughout the thesis $\odot$ denotes multiplication in $\mathbb{F}_2$; we might sometimes use $\cdot$ for the same operation.

Acyclicity is what makes a circuit computable. It lets us list the wires in a topological order $w_1, \dots, w_m$ in which the two operands of every gate appear before the gate itself, so that $j, k < i$. Fixing a value in $\mathbb{F}_2$ for every input gate — a constant gate carries its own label — then determines the value of every wire by induction along this order. When the inputs are clear from context we write simply w_i for this value.

Notation 2.1. For a wire w_i , $\text{vars}(w_i) \subseteq \mathcal{R}$ is the set of random inputs its defining sub-circuit depends on: $\text{vars}(w_i) = \{w_i\}$ if $w_i \in \mathcal{R}$, $\text{vars}(w_i) = \emptyset$ if $w_i \in \mathcal{S} \cup \mathcal{P}$ or w_i is a constant gate, and $\text{vars}(w_i) = \text{vars}(w_j) \cup \text{vars}(w_k)$ for a gate with operands w_j, w_k .

Two conventions, fixed here once, are used throughout. First, a topological order realises the circuit as a *straight-line program*: each wire is defined at most once, in

terms only of wires already defined, so there is no branching and no recomputation of a gate under a different name. We pass freely between the digraph of Definition 2.1 and this sequential reading, whichever is clearer at the moment. Second, the restriction to 2-ary gates is a modelling choice, not a loss of generality: an n -ary sum or product is always expressible as a chain of 2-ary gates. Forcing this decomposition up front matters for Section 2.3.2, where every gate output is a wire an adversary may separately observe — collapsing several operations into one hidden, unobservable step would understate what a probing adversary can see. Chapter 5 (Section 5.2.1) later represents this same object as a hash-consed data structure suited to computation; nothing here depends on that choice of representation.

2.2 Secret Sharing and Masking

filler

2.2.1 Boolean Masking

filler

2.3 The ISW Probing Model

filler

2.3.1 t -Threshold Probing and Observation Sets

filler

2.3.2 d -Probing Security

filler

2.3.3 Probabilistic Non-Interference

filler

2.3.4 Threshold and Strong Non-Interference

filler

2.4 Randomness Substitution

This section isolates, as a single reusable fact, the probabilistic argument our checker automates. Fix a circuit as in Section 2.1.3, with random inputs $\mathcal{R}$, and let a *randomness assignment* be a function $\rho : \mathcal{R} \rightarrow \{0, 1\}$. The secrets and public inputs are held fixed throughout, so every wire w denotes a function $w(\rho)$ of ρ alone. A *probe* is a tuple of wires — the values an adversary observes — and Section 2.3.2 phrases d -probing security as the demand that the joint distribution of a probe of size at most d , taken over the uniformly distributed ρ , not move as the secrets vary.

The whole argument rests on one line of $\mathbb{F}_2$ arithmetic: for any fixed bit c , the translation $b \mapsto c \oplus b$ is a **bijection** of $\{0, 1\}$ that preserves the uniform distribution (i.e. it is its own inverse and merely swaps the two outcomes). Everything below is this fact, applied to a single coordinate of ρ with every other coordinate frozen. The object that carries it is a substitution.

Definition 2.2 (Randomness substitution). *Let $r \in \mathcal{R}$ and let e be a wire with $r \notin \text{vars}(e)$. The substitution $\sigma = (r \mapsto e \oplus r)$ acts on a wire w by replacing every occurrence of the input r with the wire $e \oplus r$; we write the resulting wire $w\sigma$. The*

same σ acts on randomness assignments, through the map $\hat{\sigma} : \{0,1\}^{\mathcal{R}} \rightarrow \{0,1\}^{\mathcal{R}}$ given by

$$\hat{\sigma}(\rho)(r) = e(\rho) \oplus \rho(r), \quad \hat{\sigma}(\rho)(r') = \rho(r') \quad \text{for } r' \neq r.$$

Geometrically, $\hat{\sigma}$ is a *shear* of the cube $\{0,1\}^{\mathcal{R}}$: it fixes every coordinate but r and offsets that one by $e(\rho)$, a quantity that reads only the frozen coordinates. Restricted to a fibre — a pair of points differing solely in the r -coordinate — it is exactly the translation $\rho(r) \mapsto e(\rho) \oplus \rho(r)$ of the one-line fact above, so what measure it preserves it preserves by translation invariance, fibre by fibre. We call any such map — one coordinate translated by a context depending on the others, everything else left alone — a *shear*, and use the word throughout.

Pairing the two actions under one name is the point of the lemma below: the syntactic rewrite $w \mapsto w\sigma$ and the reparametrisation $\rho \mapsto \hat{\sigma}(\rho)$ are one operation seen from two sides, and $\hat{\sigma}$ does not disturb the distribution.

Lemma 2.1 (Substitution). *For a substitution $\sigma = (r \mapsto e \oplus r)$ as in Definition 2.2, the map $\hat{\sigma}$ is a measure-preserving involution of the uniform distribution on $\{0,1\}^{\mathcal{R}}$, and for every wire w ,*

$$(w\sigma)(\rho) = w(\hat{\sigma}(\rho)) \quad \text{for every randomness assignment } \rho.$$

Proof. Fix every coordinate of ρ other than r . Since $r \notin \text{vars}(e)$, the value $e(\rho)$ depends only on these frozen coordinates, so it is constant as $\rho(r)$ ranges over $\{0,1\}$. On this fibre $\hat{\sigma}$ acts by the translation $\rho(r) \mapsto e(\rho) \oplus \rho(r)$, which is measure-preserving and self-inverse by the observation above, while every other coordinate is left fixed. Hence $\hat{\sigma}$ is an involution of $\{0,1\}^{\mathcal{R}}$ preserving the uniform measure coordinatewise, and so preserving the product measure.

The identity is a structural induction on w . For $w = r$ both sides equal $e(\rho) \oplus \rho(r)$. For an input $w \neq r$ both sides equal $w(\rho)$, since $\hat{\sigma}$ fixes every coordinate but r . For a gate $w_i = w_j \oplus w_k$ or $w_i = w_j \odot w_k$, apply the hypothesis to w_j and w_k and use that $\oplus$ and $\odot$ are computed pointwise from their operands' values. $\square$

Corollary 2.1 (Single-occurrence discharge). *Suppose r occurs exactly once among a probe $\mathbf{w}$ and the wires it depends on, as an operand of a gate $x = e \oplus r$ with $r \notin \text{vars}(e)$, and let $\sigma = (r \mapsto e \oplus r)$. Then $\mathbf{w}\sigma$ is identically distributed to $\mathbf{w}$ over ρ uniform, and every occurrence of x in $\mathbf{w}\sigma$ has collapsed to the bare input r .*

Proof. Distributional equality is Lemma 2.1 applied coordinatewise to $\mathbf{w}$, using that $\hat{\sigma}$ preserves the uniform distribution on $\{0,1\}^{\mathcal{R}}$. For the second claim, $x\sigma = e \oplus (e \oplus$

$r) = r$ by $\mathbb{F}_2$ -cancellation; since r occurs nowhere else by hypothesis, no other part of $\mathbf{w}$ is touched. $\square$

Remark 2.1. *Rewriting a probe by a substitution — whether in the general form of Lemma 2.1 or the single-occurrence special case of Corollary 2.1 — leaves its distribution untouched, hence leaves untouched whether it depends on the secrets. That is the whole lever: to decide security of a probe it is enough to rewrite it, one substitution at a time, into a form with no secret in sight. The single-occurrence case is the one the **maskVerif** tool (Barthe et al., 2019) calls optimistic sampling, and the name is apt — Corollary 2.1 never inspects the content of e , firing whether or not e depends on the secrets, and checking only the syntactic pattern that r occurs once, additively. A derivation is nothing more than such substitutions chained together; how that chain is searched for, and what the composite of its $\hat{\sigma}$ witnesses, is taken up in the later chapters.*

2.5 Computational Hardness Preview

filler

3. RELATED WORK

3.1 Language-Based Verification

3.1.1 Verified Proofs of Higher-Order Masking

3.1.2 Large Sets and the Exploration Algorithm

3.1.3 Witness Replay and Derivation Reuse

3.1.4 MaskVerif With Physical Defaults

3.2 Fourier-Analytic Verification

3.2.1 The Xiao-Massey Criterion and Approximations

3.2.2 SAT-Based Realization and Its Cost

3.3 Other Approaches

3.3.1 Model Counting and SMT

3.3.2 Direct Linear-Algebraic Methods

4. COUPLINGS AND PROBING SECURITY

4.1 The Counting Problem Is #P-Complete

4.1.1 From Independence to Distribution Counting

4.1.2 #P-Completeness and Its Consequences for Verification

4.2 Couplings of Probabilistic Programs

4.2.1 Coupling Two Runs Under Different Secrets

4.2.2 Coupling Functions: Randoms to Randoms

4.3 Probing Security via Couplings

4.3.1 d -Probing Security Implies a (Family of) Coupling Functions

4.3.2 Identity-On-Leakage Couplings and Secret-Indistinguishability

4.4 Optimistic Sampling as Coupling Construction

4.4.1 Rewrites as Bijections on the Random Space

5. DESIGN OF XXX

5.1 Pipeline Overview

filler

5.2 Expression Representation

filler

5.2.1 A Hash-Consed n -Ary Circuit DAG

filler

5.2.2 Canonicalization: Flattening and Cancellation

filler

5.2.3 Multiplicative Factoring

filler

5.2.3.1 The Factoring Transformation

filler

5.2.3.2 Enlarging the Class of Verifiable Circuits

filler

5.2.3.3 Cost Analysis and the Factoring–Speed Tension

filler

5.3 Component I: The Rewrite Engine

Given a chosen probe $\mathbf{w} = (w_1, \dots, w_k)$, $k \leq d$, represented as node IDs into the hash-consed circuit DAG of Section 5.2.1, the rewrite engine decides whether $\mathbf{w}$ is secret-independent. It does so constructively: rather than reasoning about probability distributions directly, it searches for a finite sequence of applications of Lemma 2.1 (Section 2.4) after which $\mathbf{w}$'s image is *syntactically* free of secret leaves, i.e. no secret leaf is reachable in the sub-DAG below any w_i . Termination with a secret-free image certifies $\mathbf{w}$; Theorem 5.1 in Section 5.3.3 makes precise why this certificate is sound.

The engine has three tiers of increasing generality and cost, summarized in Al-

gorithm 1 and mirrored in the structure of this section. **SIMPLEREWRITE** (Section 5.3.1) handles the common case, every random occurring at most once, with a cheap, purely reference-counted procedure that never builds a new DAG node. It is tried first on the unfactored probe and, on failure, once more after the multiplicative factoring of Section 5.2.3 has had a chance to expose additional single-occurrence randoms. **GENERALREWRITE** (Section 5.3.2) is the complete fallback for the remaining case, a random occurring more than once: it substitutes a random by its context wherever the random occurs, interleaving factoring lazily whenever both of its own find-rules stall. Section 5.3.3 closes the section by reading the whole derivation as an explicit, replayable bijection on the randomness assignment, the Φ that Component III (Section 5.5) later replays against every other wire in the circuit.

5.3.1 The Simple Rewrite Rule

The cheapest tier discharges the pattern Corollary 2.1 addresses directly: a random that occurs exactly once in the sub-DAG below the probe, as a child of an XOR node. A single depth-first traversal from the probe roots computes, for every node n reachable from them, four pieces of bookkeeping: $\text{totalParCount}(n)$, the number of edges into n from within the explored region; $\text{xorParCount}(n) \leq \text{totalParCount}(n)$, the number of those edges whose source is an XOR node; $\text{mulDepth}(n)$, the number of AND-levels separating n from the nearest root; and $\text{parents}(n)$, the reverse adjacency needed to walk back up from a random to its context. A random r is a *simple-rule candidate* when $\text{totalParCount}(r) = \text{xorParCount}(r) = 1$ and r is not itself a probe root: this is Corollary 2.1’s hypothesis, checked without ever constructing e . Write x for r ’s unique parent, so $x = e \oplus r$ for the (unconstructed) context e .

A probe root is excluded from candidacy because its value is what the adversary observes: it is fixed, not fresh, and relabeling it would claim independence for something the tuple actually exposes. The tuple $(r, s \oplus r, r \oplus r_3)$ makes the danger concrete. Its first entry is the bare leaf r , and the first two entries XOR to the secret s , so this tuple leaks s and must be rejected. But once r_3 is discharged (it occurs once, additively, in $r \oplus r_3$), r ’s own reference count drops to one: its only remaining parent is $s \oplus r$. Without the root guard, r would now match the candidate pattern and be cascaded away, relabeling the very value the adversary observes and reporting a false **SECURE**.

Algorithm 1 Discharging a probe (Component I)

```
1: function CHECKPROBE( $\mathbf{w}$ )
Require: probe  $\mathbf{w}$ , node IDs into the circuit DAG  $D$  (Section 5.2.1)
Ensure: SECURE, or INSECURE( $\mathbf{w}$ )
2:   ( $ok, T$ )  $\leftarrow$  SIMPLEREWRITE( $\mathbf{w}$ ) ▷ Tier 1: unfactored
3:   if  $ok$  then return SECURE
4:   end if
5:   ( $ok, T$ )  $\leftarrow$  SIMPLEREWRITE(Factor( $\mathbf{w}$ )) ▷ Tier 2: factor (§5.2.3), retry
6:   if  $ok$  then return SECURE
7:   end if
8:   ( $ok, T$ )  $\leftarrow$  GENERALREWRITE( $\mathbf{w}, \emptyset, fuel, false, \langle \rangle$ ) ▷ Tier 3: complete
   fallback
9:   return  $ok ? \text{SECURE} : \text{INSECURE}(\mathbf{w})$ 
10: end function

11: function SIMPLEREWRITE( $roots$ )
12:    $todo \leftarrow \{r \in \text{vars}(roots) : r \text{ occurs once, additively, } r \notin roots\}$ , shallowest first
13:    $T \leftarrow \langle \rangle$ 
14:   while  $todo \neq \langle \rangle$  and not SECRETFREE( $roots$ ) do
15:      $r \leftarrow$  pop shallowest element of  $todo$ ;  $x \leftarrow$  unique additive parent of  $r$  ▷
      $x = e \oplus r$ 
16:     mark  $x$  as a fresh random leaf;  $T \leftarrow T.\text{push}(r, x)$  ▷  $t = e \oplus r = x$ , already
     in hand
17:     drop  $r$ ; decrement occurrence counts of  $x$ 's other children
18:     (a child newly matching the pattern above, and  $\notin roots$ , joins  $todo$ )
19:   end while
20:   return (SECTFREE( $roots$ ),  $T$ ) ▷ same shears, now recorded — §5.3.3
21: end function

22: function GENERALREWRITE( $tuple, used, fuel, factored, T$ )
23:   if SECRETFREE( $tuple$ ) then return ( $true, T$ )
24:   end if
25:   if  $fuel = 0$  then return ( $false, T$ )
26:   end if
27:    $r \leftarrow$  FINDSIMPLE( $tuple$ ) ▷ occurs once, additively, not a root
28:   if  $r = \perp$  then  $r \leftarrow$  FINDGENERAL( $tuple, used$ ) ▷ occurs  $\geq 2\times$ ; some
   additive parent free of  $r$ 
29:   end if
30:   if  $r = \perp$  then
31:     if  $factored$  then return ( $false, T$ )
32:     end if
33:     return GENERALREWRITE(Factor( $tuple$ ),  $used, fuel, true, T$ ) ▷ §5.2.3.3,
   lazy retry
34:   end if
35:    $x \leftarrow$  chosen additive parent of  $r$ ;  $e \leftarrow \bigoplus(\text{children}(x) \setminus \{r\})$  ▷ side cond.
    $r \notin \text{vars}(e)$ 
36:    $t \leftarrow e \oplus r$ ;  $tuple' \leftarrow tuple[r \mapsto t]$  ▷ substitute everywhere — Lemma 2.1
37:   return GENERALREWRITE( $tuple', used \cup \{r\}, fuel - 1, false, T.\text{push}(r, t)$ )
38: end function
```

Discharging a candidate (`APPLYREWRITE` in the implementation) does not rebuild x as $e \oplus r$; it marks x itself as a fresh random leaf for every purpose the bookkeeping cares about, drops r , and decrements the reference counts of x 's other children, each of which loses the one incoming edge from x . This is licensed by Lemma 2.1 applied with this specific r and e : x , jointly with everything not depending on r , is distributed exactly as fresh, independent randomness, so nothing is lost by never inspecting e 's content. Because the DAG is hash-consed (Section 5.2.1), x may have several parents; a single discharge blinds every one of them at once, since all read the identical shared node.

Decrementing a child's count can itself drop it to a value matching the candidate pattern, in which case it joins the work-list, ordered by increasing `mulDepth` so that the shallowest, closest-to-a-root candidates fire first, matching the increasing-depth discharge order used by `maskVerif` (Barthe et al., 2019). This cascade is again guarded by root membership. The loop succeeds once every probe root fails to be a bare secret leaf and every secret leaf in the circuit has zero recorded incoming edges within the current bookkeeping, a check answerable in $O(1)$ from the maintained counts, with no fresh traversal. It fails once the work-list empties without reaching that state. A random is discharged at most once, since its bookkeeping is erased the moment it fires, so the loop performs at most $|\mathcal{R}|$ rewrite steps.

This tier is sound, since each step is a direct instance of Corollary 2.1, but incomplete: it has nothing to offer a random occurring twice, such as a mask reused across two cross-terms of a masked multiplication. Section 5.3.2's fallback handles that case, at the price of the incremental, allocation-free bookkeeping this tier relies on.

5.3.2 Fallback: General Substitution

`GENERALREWRITE` handles randoms that occur more than once by performing the substitution of Lemma 2.1 literally, rather than only its single-occurrence case (Corollary 2.1). It picks a random r and an additive occurrence $x_1 = e \oplus r$ with $r \notin \text{vars}(e)$, then rewrites every root by substituting $r \mapsto e \oplus r$ throughout. Recanonicalization (Section 5.2.2) collapses the chosen occurrence x_1 to the bare leaf r by $\mathbb{F}_2$ -cancellation, exactly as in Corollary 2.1, while every other occurrence of r absorbs e instead of vanishing, cancelling further if it happens to share structure with e . The loop succeeds once no secret leaf remains reachable from the rewritten

roots.

Each round recomputes both find-rules on the tuple’s current form: a fresh depth-first traversal from the current roots, repeated every round rather than maintained incrementally as in Section 5.3.1. FINDSIMPLE looks first for the same single-occurrence, non-root pattern as Section 5.3.1, since a general substitution can turn a multi-occurrence random into a single-occurrence one; it prefers the shallowest candidate. Failing that, FINDGENERAL looks for any random, not yet used by a previous general step, occurring at least twice with some additive parent x_1 whose other children do not themselves depend on it, the side condition $r \notin \text{vars}(e)$ that Lemma 2.1 requires. It prefers the deepest candidate, the opposite bias from FINDSIMPLE; the implementation records no rationale for this choice beyond the ordering itself. A context e is always read off an XOR node’s other children: anything else would mean e contains r , making $r \mapsto e \oplus r$ fail to be a bijection, so the implementation fails loudly there instead of silently admitting an unsound relabeling. The same traversal answers whether the tuple is secret-free, since a secret survives exactly when it is itself a root or has a recorded parent in the explored sub-DAG, so no second traversal is needed for that check.

When both find-rules stall on the tuple’s current form, the engine factors (Section 5.2.3) once and retries, rather than declaring failure immediately. Factoring can move a random from a purely multiplicative position, where neither find-rule can see it, into an additive one, but on its own it eliminates no secret, so it must be interleaved with substitution rather than run once up front or once at the end. Factoring is symmetrically not tried before the simple rule either: Algorithm 1 always tries SIMPLEREWRITE unfactored first, because factoring can destroy a single-occurrence opportunity the unfactored form exposes, so trying the cheap, unfactored tier first is not just faster on the common case but strictly more complete.

FINDGENERAL is needed precisely when a random occurs additively in two places that do not telescope into one another under the simple rule. Suppose r occurs as an operand of $x_1 = e_1 \oplus r$ in one root and of $x_2 = e_2 \oplus r$ in another, with $r \notin \text{vars}(e_1) \cup \text{vars}(e_2)$. Substituting through x_1 , i.e. $r \mapsto e_1 \oplus r$, turns x_1 into $e_1 \oplus (e_1 \oplus r) = r$ by cancellation, as in Corollary 2.1, while the same substitution turns x_2 into $e_2 \oplus e_1 \oplus r$. If $e_1 \oplus e_2$ is itself secret-free, both roots are secret-free after this one step; otherwise the loop continues, substituting further or factoring to expose a new opportunity. This is the shape of the case a masked multiplication typically produces, where a single random masks two different cross-terms: neither term’s random is disposable on its own, but the substitution above can still cancel the secret-dependent part they share. Whether a given instance is discharged this way,

and in how many rounds, depends entirely on how much of $e_1 \oplus e_2$ the rest of the derivation can still cancel; the loop offers no guarantee beyond trying. Even granting unlimited fuel, it only composes single-coordinate shears $\hat{\sigma}$, so it can only certify secret-independence witnessed by a chain of substitutions, possibly interleaved with factoring. Completeness is proved only in the linear/affine case (nerede proof?); whether a genuinely irreducible counterexample exists beyond it is not settled by this thesis.

Termination is only argued informally, not proved. Each general substitution retires its random into *used* and is never repeated for it, bounding the number of general steps by $|\mathcal{R}|$; between general steps, simple substitutions strictly reduce the candidate pool; factoring is retried at most once per stall before the engine gives up on the current form. Interleaving factoring with two mutually triggering find-rules resists a clean potential-function argument, so a generous fuel counter backstops the loop instead. Exhausting fuel is treated as failure, which is sound (Theorem 5.1 is never invoked on a fuel-out) but not necessarily tight: a fuel-out reports the probe as though it were genuinely insecure, without exhibiting an actual distinguishing observation.

5.3.3 Relation to the Coupling-Construction View

A discharge does more than answer SECURE/INSECURE. Read the right way, it hands back the object Chapter 4 asks for: a coupling for the probe. We state that object declaratively first, then show how the engine produces one.

Let the secrets range now, rather than staying fixed as in Section 2.4: write $\mathbf{w}(\rho; \mathbf{a})$ for the value of the probe under randomness assignment $\rho \in \{0,1\}^{\mathcal{R}}$ and secret assignment $\mathbf{a} \in \{0,1\}^{\mathcal{S}}$, the public inputs held fixed.

Definition 5.1 (Coupling of a probe). *A coupling of a probe $\mathbf{w}$ is a family $\{\Phi_{\mathbf{a},\mathbf{b}}\}_{\mathbf{a},\mathbf{b} \in \{0,1\}^{\mathcal{S}}}$ of measure-preserving bijections of $\{0,1\}^{\mathcal{R}}$, one per ordered pair of secret assignments, satisfying*

$$\mathbf{w}(\rho; \mathbf{a}) = \mathbf{w}(\Phi_{\mathbf{a},\mathbf{b}}(\rho); \mathbf{b}) \quad \text{for every } \rho \in \{0,1\}^{\mathcal{R}}.$$

Definition 5.1 says precisely where to send the randomness so that a run under $\mathbf{a}$ is reproduced, observation for observation, by a run under $\mathbf{b}$: each $\Phi_{\mathbf{a},\mathbf{b}}$ dictates

the relabelling of the random tape that absorbs the change of secret. Since it is measure-preserving, pushing the uniform tape through it turns this pointwise identity into an equality of laws, so the distribution of $\mathbf{w}(\cdot; \mathbf{a})$ matches that of $\mathbf{w}(\cdot; \mathbf{b})$. A coupling existing for *every* pair $(\mathbf{a}, \mathbf{b})$ is thus the same statement as $\mathbf{w}$ being secret-independent — which is what makes it the right certificate. The family carries the structure one expects of such a certificate: one may take $\Phi_{\mathbf{a}, \mathbf{a}} = \text{id}$, $\Phi_{\mathbf{b}, \mathbf{a}} = \Phi_{\mathbf{a}, \mathbf{b}}^{-1}$, and $\Phi_{\mathbf{b}, \mathbf{c}} \circ \Phi_{\mathbf{a}, \mathbf{b}} = \Phi_{\mathbf{a}, \mathbf{c}}$, so a single anchor $\Phi_{\mathbf{a}_0, \cdot}$ already generates all of it. It is these properties we care about, not any one procedure that realises them.

What the engine records is one such procedure. Discharging a probe amounts to finding a *derivation*: a finite sequence of rewrites

$$\sigma_1 = (r_1 \mapsto e_1 \oplus r_1), \dots, \sigma_K = (r_K \mapsto e_K \oplus r_K),$$

each valid — $r_i \notin \text{vars}(e_i)$ — against the tuple's state just before step i , after which no secret leaf remains reachable. A rewrite is a rewrite: whatever rule proposed it, and whether the random it retires occurred once or many times, each σ_i is a substitution in the sense of Definition 2.2, hence a single measure-preserving shear $\widehat{\sigma}_i$ (Lemma 2.1). The coupling is assembled from these shears and from nothing else, so the construction below never needs to know how any individual rewrite was found.

Theorem 5.1 (Soundness of discharge). *If $\text{CHECKPROBE}(\mathbf{w})$ (Algorithm 1) returns SECURE, then the joint distribution of $\mathbf{w}(\rho; \mathbf{a})$, for ρ sampled uniformly at random, does not depend on the secret assignment $\mathbf{a}$. Moreover the recorded substitutions realise a coupling of $\mathbf{w}$ in the sense of Definition 5.1.*

Proof sketch. Fix a secret assignment $\mathbf{a}$. Each recorded step contributes the tape shear $\widehat{\sigma}_i$, whose defining context e_i may itself read the secrets, so write $\widehat{\sigma}_i^{\mathbf{a}}$ for its action under $\mathbf{a}$ and set

$$\varphi_{\mathbf{a}} = \widehat{\sigma}_1^{\mathbf{a}} \circ \widehat{\sigma}_2^{\mathbf{a}} \circ \dots \circ \widehat{\sigma}_K^{\mathbf{a}},$$

a composition of measure-preserving bijections, hence itself one. Writing $\mathbf{w}_K = \mathbf{w} \sigma_1 \dots \sigma_K$ for the final tuple and chaining Lemma 2.1 through the K steps gives $\mathbf{w}(\varphi_{\mathbf{a}}(\rho); \mathbf{a}) = \mathbf{w}_K(\rho)$ for every ρ . A SECURE verdict means $\mathbf{w}_K$ is syntactically secret-free — no secret leaf is reachable from it — so $\mathbf{w}_K(\rho)$ does not depend on $\mathbf{a}$. The right-hand side is therefore common to all secret assignments, and for any two, $\mathbf{a}$ and $\mathbf{b}$,

$$\mathbf{w}(\varphi_{\mathbf{a}}(\rho); \mathbf{a}) = \mathbf{w}_K(\rho) = \mathbf{w}(\varphi_{\mathbf{b}}(\rho); \mathbf{b}).$$

Substituting $\rho \mapsto \varphi_{\mathbf{a}}^{-1}(\rho)$ and setting $\Phi_{\mathbf{a}, \mathbf{b}} = \varphi_{\mathbf{b}} \circ \varphi_{\mathbf{a}}^{-1}$ gives $\mathbf{w}(\rho; \mathbf{a}) = \mathbf{w}(\Phi_{\mathbf{a}, \mathbf{b}}(\rho); \mathbf{b})$

for all ρ ; the family $\{\Phi_{\mathbf{a},\mathbf{b}}\}$ is the promised coupling, and measure-preservation of each member yields the distributional claim. The chaining uses only that each σ_i is a substitution (Definition 2.2); a single-occurrence rewrite, where e_i is read off one $\oplus$ -parent as in Corollary 2.1, is folded in exactly like any other, so the argument is indifferent to how the derivation was produced. $\square$

The composite $\varphi_{\mathbf{a}}$ is displayed above as a product of shears only for exhibition; it is never used. Component III (Section 5.5.2) uses a single member of the family as a *blinding certificate*: it takes any other wire u in the circuit and admits it into the certified *safe set* when $u \circ \varphi_{\mathbf{a}}$ is secret-free — when the same relabelling that blinds the probe blinds u too. Evaluating $\varphi_{\mathbf{a}}$ on a sampled tape is a back-substitution through the recorded steps in reverse order $i = K, \dots, 1$, since the net image of r_i is $t_i = e_i \oplus r_i$ evaluated against the assignment already updated by every later step: the derivation performs forward elimination, and reading the net map off is back-substitution through the same steps in reverse, exactly as with a triangularised linear system.

The recorded target of a rewrite costs nothing to keep. The substitution $\sigma_i = (r_i \mapsto e_i \oplus r_i)$ sends r_i to $t_i = e_i \oplus r_i$, and t_i is exactly the $\oplus$ -parent node the engine already held when it chose the rewrite — there is no separate context e_i to reconstruct. So every rewrite the engine makes, of whatever origin, can be appended to the derivation $\sigma_1, \dots, \sigma_K$ as it happens, and the coupling is read off that one list. Component III replays these shears directly against every other wire in the circuit; no probe and no rewrite is special-cased, and each step that helped certify the probe is a step that helps blind its neighbours.

5.4 Component II: Traversing the Subset Space

filler

5.4.1 The Space-Splitting Worklist Representation

filler

5.4.2 Large-Set Pruning and Closure-Based Splitting

filler

5.4.3 Completeness via Vandermonde's Identity

filler

5.5 Component III: Extending Probe Sets

filler

5.5.1 Closures: Structural Deduction of Free Wires

filler

5.5.2 Coupling-Driven Extension

filler

6. IMPLEMENTATION OF XXX

7. EVALUATION

7.1 Benchmarks

7.2 Methodology and Metrics

7.3 Results

7.3.1 Verification Outcomes and Case Studies

7.3.2 Redundancy in the Search

7.3.3 The Cost and Benefit of Extending Probe Sets

7.4 Discussion

8. FUTURE WORK

8.1 Arithmetic Circuits

8.2 Symmetry-Orbit Space Exploration

8.3 Randomized Fast-Reject for the Engine

8.4 Scaling and Compositional Verification

8.5 Glitches and Practical Aspects

9. CONCLUSION

BIBLIOGRAPHY

- Barthe, G., Belaïd, S., Cassiers, G., Fouque, P.-A., Grégoire, B., & Standaert, F.-X. (2019). Maskverif: Automated verification of higher-order masking in presence of physical defaults. In K. Sako, S. A. Schneider, & P. Y. A. Ryan (Eds.), *Computer security – esorics 2019* (pp. 300–318, Vol. 11735). Springer International Publishing. https://doi.org/10.1007/978-3-030-29959-0_15

APPENDIX